

Name - Chaitanya Kelkar

Roll no - 15

Experiment No 6

Aim

Creation and Deployment of Smart Contract and Transactions using Solidity and Remix IDE.

Theory

1. Relevance of require Statements

In Solidity, the require statement serves as a guard condition inside functions to enforce execution rules and input parameters before executing contract logic.

- **State Rollback:** If the assertion inside a require statement evaluates to false, function execution terminates immediately, and all modifications to state variables and Ether balances revert to their state prior to transaction execution.
- **Access Control & Integrity:** It ensures that only authorized entities (e.g., verifying `msg.sender == chairperson`) or valid states (e.g., verifying a voter has not already voted) can execute sensitive operations, preventing corrupted state writes on the blockchain.

2. Keywords: mapping, storage, and memory

- **mapping:** A key-value hash table data structure declared as `mapping(keyType => valueType)`. It is used to associate keys (such as Ethereum addresses) with values (such as custom Voter structs). Mappings cannot be enumerated and do not store array lengths, making lookups highly gas-efficient.
- **storage:** Refers to permanent variable storage maintained on the Ethereum blockchain. State variables declared at the contract level are stored in storage by default and persist across transactions and contract calls. Modifying storage consumes significant gas due to permanent state writes.
- **memory:** Refers to temporary data storage that exists only during the lifetime of a function call. Data in memory is erased once function execution finishes, making it ideal for temporary computation parameters, string processing, and function arguments at a lower gas cost.

3. Trade-offs: bytes32 vs. string

- **bytes32 (Fixed Size):** Allocates a fixed 32-byte slot in EVM storage. Operations on bytes32 execute faster and consume significantly less gas, but limit input length to 32 characters.
- **string (Dynamic Size):** Represents dynamic UTF-8 character sequences. While dynamic typing simplifies user interaction and parameter passing on client applications, EVM overhead for handling variable lengths increases gas costs. Updating modern contracts from bytes32 to string prioritizes readability and user experience.

Modified Ballot.sol Source Code and Successful Compilation (bytes32 to string)

```
1 // SPDX-License-Identifier: GPL-3.0
2 pragma solidity >=0.7.0 <0.9.0;
3
4 /**
5  * @title Ballot
6  * @dev Implements voting process along with vote delegation
7  */
8 contract Ballot {
9     struct Voter {
10         uint weight; // weight is accumulated by delegation
11         bool voted; // if true, that person already voted
12         address delegate; // person delegated to
13         uint vote; // index of the voted proposal
14     }
15
16     struct Proposal {
17         string name; // Changed from bytes32 to string for readability
18         uint voteCount; // number of accumulated votes
19     }
20
21     address public chairperson;
22     mapping(address => Voter) public voters;
23     Proposal[] public proposals;
24
25     /**
26     * @dev Create a new ballot to choose one of 'proposalNames'
27     * @param proposalNames names of proposals as strings
28     */
29 }
```

Smart Contract Deployment with Constructor Parameters (["Chaitanya", "Tejas", "Akshay"])

DEPLOY & RUN TRANSACTIONS

Environment

Fork

Reset

Remix VM

Osaka

Account 1

0x5B3...eddC4

100.000 ETH

Deploy

Deployed Contracts

Transaction History

Deploy

Remix VM osaka

Ballot

3_Ballot.sol

✓ Compiled

proposalNames

string[]

["Chaitanya","Tejas","Akshay"]

Call data

Params

Auto-Fill with AI Samples

Value

0

wei

Granting Voting Rights (giveRightToVote) and Viewing Initial Proposal State (proposals(0))

[illegible]

● **giveRightToVote** (address)

0xAb8483F64d9C6d1EcF9b849Ae677dD33

Transact

Calldata  Params 

✓ [vm] from: 0x5B3...eddC4 to: Ballot.giveRightToVote(address) 0xd91...39138 value: 0 wei data: 0x9e7...35cb2 logs: 0 hash: 0x3d0...3c30d Debug	
status	1 Transaction mined and execution completed
transaction hash	0x3d03f2e54a12506df2407a1f5036505dd2b8447d86b0dca1fc71a1cf17e3c30d
block hash	0x38746d71e5c88f57b7b174bc9b3d998fa5760f0deea07be47eefa6c0ee459dea
block number	6
from	0x5B38Da6a701c568545dCfcB03FcB875F56beddC4
to	Ballot.giveRightToVote(address) 0xd9145CCE52D386f254917e481eB44e9943F39138
transaction cost	48642 gas
execution cost	27210 gas
output	0x
decoded input	<pre>{ "address voter": "0xAb8483F64d9C6d1EcF9b849Ae677dD3315835cb2" }</pre>

Executing Vote Transaction (vote(2)) and Retrieving Winner Results (winningProposal & winnerName)

● **vote (uint256)**

Transact

Calldata
Params

✓ [vm] from: 0xAb8...35cb2 to: Ballot.vote(uint256) 0xd91...39138 value: 0 wei data: 0x012...00002 logs: 0 hash: 0x0b6...33816	
status	1 Transaction mined and execution completed
transaction hash	0x0b6f332a7581b5aa59e1fb7055720a7426fa05938ada6f2a570539bd14233816
block hash	0x5d12d40b4fb43cec7dcb8309b0b8a86a05b021518f25937bece28326f544e653
block number	7
from	0xAb8483F64d9C6d1EcF9b849Ae677dD3315835cb2
to	Ballot.vote(uint256) 0xd9145CCE52D386f254917e481eB44e9943F39138
transaction cost	92896 gas
execution cost	71692 gas
output	0x
decoded input	<pre>{ "uint256 proposal": "2" }</pre>

● winnerName

Call

● winningProposal

Call

0:uint256: winningProposal_ 2

```
CALL [call] from: 0xAb8483F64d9C6d1EcF9b849Ae677dD3315835cb2 to: Ballot.winnerName() data: 0xe2b...a53f0
```

```
CALL [call] from: 0xAb8483F64d9C6d1EcF9b849Ae677dD3315835cb2 to: Ballot.winningProposal() data: 0x609...ff1bd
```

Conclusion

In this experiment, the Ballot smart contract was updated from `bytes32` to `string` proposal parameters, compiled, and deployed on the Remix VM environment. The deployment confirmed that `string` proposal arrays can be passed directly into the contract constructor and retrieved as readable text values. All core voting workflows—including chairperson privilege authorization, vote delegation, proposal voting, and automated winner calculation—were executed successfully with transaction logs verified on the EVM state.